

PROJECTGROUP_4 BEGINNER GUIDE TO streamlit_app.py

Purpose

This document explains the current streamlit_app.py file from top to bottom. The source file is 177 lines long. Blank lines are included only to make the source easier to read; they do not perform an action. The explanations below cover every non-blank line or closely related line group.

What the program does

The program models a small web search engine. It stores eight fictional pages, calculates PageRank from the links between those pages, matches a user's search words, combines relevance with authority, and displays the result in a browser. Streamlit lets the interface be written in Python.

How to run it

From the ProjectGroup_4 folder:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run streamlit_app.py
```

Open the local address printed by Streamlit, usually <http://localhost:8501>.

PART 1: IMPORTS AND BASIC SETUP

=====

Line 1: `from __future__ import annotations`

This tells Python to handle type annotations in a flexible way. A type annotation describes what kind of value a function expects or returns. This line is optional for the algorithm, but makes annotations easier to use.

Line 3: `import re`

This imports Python's regular-expression module. The app uses it later to split search text into simple words.

Line 5: `import streamlit as st`

This imports the Streamlit library and gives it the short name `st`. Every statement beginning with `st.` asks Streamlit to create part of the interface, such as a title, slider, table, or graph.

PART 2: THE MINIATURE WEB DATA

=====

Line 8: `PAGES = [`

This starts a Python list named `PAGES`. A list is an ordered collection. Here, each item in the list represents one web page in the model.

Lines 9-15: first page dictionary

The braces create a dictionary: a collection of named values.

Line 10: `"id": "atlas"`

This gives the page a short internal identifier. Other pages use this value when they create links.

Line 11: `"title": "The Atlas of the Open Web"`

This is the human-readable page title shown in search results.

Line 12: `"domain": "atlas.example"`

This is an example website address. It is fictional data, not a real website.

Line 13: "text": "..."

This is the page description. The search function checks this text for query words.

Line 14: "links": [...]

This is a list of page IDs that atlas links to. These links form the directed graph used by PageRank.

Lines 16-22: second page dictionary

These lines define the ranking page in the same format: an ID, title, domain, description, and four outgoing links. Reusing the same structure makes all pages easy for the algorithm to process.

Lines 23-29: third page dictionary

These lines define the algorithms page. It links to ranking, search, and code. A link is represented by a string matching another page's id.

Lines 30-36: fourth page dictionary

These lines define the protocols page, titled How the Web Works. Its links connect it to atlas, code, and libraries.

Lines 37-43: fifth page dictionary

These lines define the commons page and its links to atlas, libraries, and search.

Lines 44-50: sixth page dictionary

These lines define the libraries page and its links to code, commons, and algorithms.

Lines 51-57: seventh page dictionary

These lines define the search page. It describes crawling, indexing, matching, and ranking, and links to ranking, algorithms, and code.

Lines 58-64: eighth page dictionary

These lines define the code page. It links to algorithms, libraries, and protocols.

Line 65:]

This closes the PAGES list. At this point the program has an eight-node web graph with 26 total outgoing links.

PART 3: CALCULATING PAGERANK

=====

Line 68: @st.cache_data

This is a decorator. A decorator changes how the function below is handled. Streamlit remembers results for inputs it has already seen, so it can avoid repeating the same calculation unnecessarily.

Line 69: def calculate_pagerank(damping: float, iterations: int) -> dict[str, float]:

This defines a function named calculate_pagerank.

The function receives two inputs:

- damping is a decimal such as 0.85.
- iterations is a whole number such as 20.

The return annotation says the function returns a dictionary whose keys are strings and whose values are decimal numbers.

Line 70: `total_pages = len(PAGES)`
len counts how many items are in PAGES. Here total_pages becomes 8.

Line 71: `scores = {page["id"]: 1 / total_pages for page in PAGES}`
This creates the initial score for every page. Every page starts equally, so each receives 1 divided by 8. The expression is a dictionary comprehension, a compact way to build a dictionary by looping through PAGES.

Line 73: `for _ in range(iterations):`
This repeats the ranking calculation. `range(iterations)` creates the requested number of rounds. The underscore means the loop number itself is not needed.

Lines 74-76: create next_scores
Every new score starts with $(1 - \text{damping})$ divided by total_pages.
With damping 0.85, this is the small amount of authority distributed evenly to every page. This prevents a page from receiving zero authority forever.

Line 77: `for page in PAGES:`
This loops through every page and asks it to distribute its current authority.

Line 78: `share = scores[page["id"]] / max(len(page["links"]), 1)`
The page's score is divided among its outgoing links. `max(..., 1)` prevents division by zero if a page has no links.

Line 79: `for target in page["links"]:`
This loops over each destination linked from the current page.

Line 80: `next_scores[target] += damping * share`
The destination receives the source page's share multiplied by the damping factor. This is the main authority-transfer step in PageRank.

Line 81: `scores = next_scores`
After one round, the new scores become the scores used by the next round.

Line 83: `return scores`
After all iterations finish, the function gives the final authority dictionary back to the caller.

The PageRank idea in one formula

For a destination page A:

$$PR(A) = (1-d)/N + d * \sum(PR(B) / L(B))$$

N is the number of pages. d is the damping factor. B is a page linking to A.
L(B) is the number of links leaving B. Important pages pass more authority to the pages they link to.

PART 4: TURNING SEARCH TEXT INTO WORDS

=====

Line 86: `def tokenize(value: str) -> list[str]:`
This defines tokenize. It accepts a string and returns a list of strings.

Line 87: `return re.findall(r"[a-z0-9]+", value.lower())`
`value.lower()` converts uppercase letters to lowercase. The regular expression finds groups of letters and numbers. For example, "Web Search!" becomes ["web", "search"]. This makes matching simpler and ignores punctuation.

PART 5: SEARCHING AND COMBINING SCORES

=====

Line 90: `def search_pages(query: str, damping: float, iterations: int) -> list[dict]`:

This defines the search function. It accepts the user's query and the two PageRank controls, then returns a list of result dictionaries.

Line 91: `ranks = calculate_pagerank(damping, iterations)`

The search always uses current PageRank scores. If the user changes a slider, this call uses the new values.

Line 92: `maximum_rank = max(ranks.values())`

This finds the largest authority score. It is used to normalize other scores so the strongest page has authority 1.0, or 100 percent.

Line 93: `terms = tokenize(query)`

The user's search phrase is converted into a list of lowercase search terms.

Line 94: `results = []`

This creates an empty list. Matching pages will be added to it.

Line 96: `for page in PAGES:`

The function examines each page one at a time.

Line 97: `words = tokenize(f'{page["title"]} {page["text"]} {page["domain"]}')`

The title, description, and domain are joined into one string and split into words. These are the words that can match the query.

Line 98: `hits = sum(words.count(term) for term in terms)`

For every query term, count how often it appears in the page words. `sum` adds those counts together. More matching words mean stronger relevance.

Line 99: `relevance = min(hits / len(terms), 1) if terms else 0`

If the query has words, `hits` is divided by the number of query terms. `min` prevents relevance from going above 1. If the query is empty, relevance is 0.

Line 100: `authority = ranks[page["id"]] / maximum_rank`

This converts the PageRank score into a relative authority from 0 to 1.

Line 101: `if relevance or not terms:`

Only pages with a keyword match are shown for a non-empty query. If the query is empty, all pages are allowed through.

Lines 102-107: `results.append({...})`

A matching page is copied into a new result dictionary. `**page` copies the original page fields. The new fields store relevance, normalized authority, and the final score.

Line 104: `"relevance": relevance`

Stores the text-match score for later display.

Line 105: `"authority": authority`

Stores the normalized PageRank score for later display.

Line 106: `"score": relevance * 0.68 + authority * 0.32`

This is the final ranking formula. Text relevance contributes 68 percent and PageRank authority contributes 32 percent.

Line 109: `return sorted(results, key=lambda page: page["score"], reverse=True)`

`sorted` orders the matching pages. `key` says to use each page's score. `reverse=True` means highest scores appear first. `lambda` is a small unnamed function.

PART 6: BUILDING THE GRAPH DESCRIPTION

=====

Line 112: `def graph_text(ranks: dict[str, float], highlighted: set[str]) -> str:`
This function creates a Graphviz text description. It receives PageRank scores and the IDs that should be highlighted, then returns graph text.

Lines 113-114: `lines = [...]`
This starts a list of Graphviz commands. The commands set the graph direction to left-to-right, make nodes circles, and choose colors for links and nodes.

Line 115: `maximum_rank = max(ranks.values())`
The largest rank is needed to scale node sizes relative to the most authoritative page.

Line 116: `for page in PAGES:`
The app creates one visual graph node per page.

Line 117: `size = 0.45 + 0.65 * ranks[page["id"]] / maximum_rank`
A page's circle gets larger when its authority is larger. The starting 0.45 ensures even low-authority pages remain visible.

Line 118: `color = ... if ... else ...`
Matching pages get coral. Other pages get lime. This connects search results to the visual graph.

Lines 119-120: `lines.append(f'...')`
This formats a Graphviz node command containing the ID, label, size, and color. The `f` before the string allows Python values to be inserted into the text.

Lines 121-123: nested loops for links
The outer loop visits each page. The inner loop visits each outgoing link. Each pair becomes a Graphviz arrow from the source page to the target page.

Line 124: `lines.append("}")`
This closes the Graphviz document.

Line 125: `return "\n".join(lines)`
The list of graph commands is joined with newline characters and returned as a single string for Streamlit.

PART 7: BUILDING THE STREAMLIT INTERFACE

=====

Lines 128-129: `st.set_page_config(...)`
These set the browser page title, the small page icon, and a wide layout.

Line 130: `st.caption(...)`
Displays small supporting text above the main title.

Line 131: `st.title(...)`
Displays the main heading.

Line 132: `st.write(...)`
Displays a short explanation of what the application does.

Line 134: `with st.sidebar:`
Everything indented below this line is placed in Streamlit's sidebar. `with` is a Python context manager that temporarily changes where UI elements are placed.

Line 135: `st.markdown("### Ranking controls")`
Displays a small Markdown heading in the sidebar.

Line 136: `damping = st.slider(...)`
Creates a slider between 0.50 and 0.99. It starts at 0.85 and moves in steps of 0.01. The selected value is stored in `damping`.

Line 137: `iterations = st.slider(...)`
Creates a whole-number slider from 1 to 50. It starts at 20. The selected value is stored in `iterations`.

Lines 138-139: `st.caption(...)`
Explains the meaning of the default damping factor to the user.

Line 140: `st.metric("Indexed pages", len(PAGES))`
Shows the number of pages in the small web graph.

Line 141: `st.metric("Known links", sum(...))`
Counts all outgoing links and shows the total in the sidebar.

Line 143: `query = st.text_input(...)`
Creates the search box and gives it a default example query. The typed text is stored in `query`.

Line 144: `active_query = query`
Makes a clearly named variable for the query that will be searched. In this simplified version it is the same value as `query`.

Line 146: `results = search_pages(active_query, damping, iterations)`
Runs the complete search process using the current input and controls.

Line 147: `ranks = calculate_pagerank(damping, iterations)`
Calculates the ranks again so the graph and score table can use them.

Line 148: `highlighted = {result["id"] for result in results[:4]}`
Creates a set containing the first four result IDs. A set is useful because it makes membership checks fast. These IDs are highlighted in the graph.

Line 150: `results_column, graph_column = st.columns([1.5, 1], gap="large")`
Creates two side-by-side areas. The results area is 1.5 parts wide and the graph area is 1 part wide.

Lines 151-152: `with results_column / st.subheader(...)`
The following result UI is placed in the left column, beginning with its heading.

Line 153: `st.caption(f'Showing matches ...')`
Shows the current query and the number of matching pages. The f-string inserts those changing values into the message.

Lines 154-155: empty-result condition
If the list is empty, `st.info` displays a friendly message instead of a blank screen.

Line 156: `for index, page in enumerate(results, 1):`
Loops through results while giving each one a human-friendly number starting at 1. Both the number and page dictionary are available inside the loop.

Line 157: `with st.container(border=True):`
Creates a bordered result box. All indented elements belong to that result.

Line 158: `st.caption(f'{index:02d} / {page["domain"]}')`
Shows the result number with two digits, such as 01, plus its example domain.

Line 159: `st.subheader(page["title"])`

Displays the page title as a result heading.

Line 160: `st.write(page["text"])`
Displays the page description.

Lines 161-165: `st.write(f"Match ...")`
Displays the relevance percentage, authority percentage, and number of outgoing links. The `:.0%` format turns a decimal such as 0.8 into 80%.

Lines 167-168: `with graph_column / st.subheader(...)`
Switches to the right column and gives the graph area a heading.

Lines 169-170: `st.caption(...)`
Explains what graph size and highlighting mean.

Line 171: `st.graphviz_chart(graph_text(ranks, highlighted), ...)`
Calls `graph_text` to build the Graphviz source, then asks Streamlit to render it as a graph. The graph uses the current PageRank scores and query highlights.

Line 172: `st.subheader("PageRank scores")`
Adds a heading above the score table.

Lines 173-174: `scores = [...] for page in sorted(...)`
Builds a list of small dictionaries for the table. `sorted` orders the pages by highest rank first. `round` keeps the displayed authority readable.

Line 175: `st.dataframe(scores, hide_index=True, use_container_width=True)`
Displays the scores as a table. The index column is hidden and the table uses the available column width.

Line 177: `st.caption(...)`
Shows a final note explaining the 68/32 scoring formula and reminding the user that this is an educational model, not a real web crawler.

THE COMPLETE USER FLOW

=====

1. Streamlit starts and runs the Python file from top to bottom.
2. `PAGES` defines the miniature web graph.
3. The sidebar collects damping and iteration settings.
4. The search box collects a query.
5. `calculate_pagerank` sends authority through the links.
6. `search_pages` measures word matches and combines both scores.
7. `graph_text` creates the graph description.
8. Streamlit displays results, graph nodes, and the authority table.
9. When a widget changes, Streamlit reruns the script with the new values.

Beginner vocabulary

=====

- Variable: a name that stores a value.
- List: an ordered collection of values.
- Dictionary: a collection of key/value pairs.
- Function: reusable code that performs a task.
- Parameter: an input accepted by a function.
- Loop: code that repeats.
- Condition: code that chooses whether something should happen.
- Return: the value a function sends back.
- Module: reusable code imported from another package or file.
- Decorator: syntax that changes or adds behavior to a function.

Important limitation

=====

The graph and pages are manually created in `PAGES`. The app does not download

real websites, crawl the internet, store user accounts, or use a production search index. Its purpose is to make the core PageRank idea easy to see.